



Hochschule für Technik,
Wirtschaft und Kultur Leipzig

Bachelor Thesis

Entwicklung einer Mobilen Applikation zur
effizienten Stimmung einer Floyd-Rose-Gi-
tarre

Name: Raphael Schütz

Matrikelnummer: 82832

Fachsemester: 7

Erstprüfer: Prof. Konrad Schöbel

Zweitprüfer: Prof. Ulf Schemmert

Abgabe: 01.03.2026

Inhaltsverzeichnis

1	Motivation	4
2	Grundlagen (Physik der Gitarre)	4
2.1	Experiment: Elastizität von Gitarrensaiten	5
2.2	Physikalisches Modell	7
2.3	Experiment: Nachweis Linearität	11
2.3.1	Vorgehensweise	11
2.3.2	Ergebnisse	12
2.3.2.1	Relative Visualisierung der Frequenzänderungen	12
2.3.2.2	Korrelationskoeffizienten und Fehler	13
2.3.3	Diskussion der Ergebnisse	13
2.4	Mathematische Lösung	14
3	Ablauf eines Stimmvorgangs	16
3.1	Initierung - Bestimmung der Verstimmungsmatrix	16
3.2	Stimmen der Gitarre	16
3.3	Überprüfung	16
4	Grundlegende Verfahren	16
4.1	Bestimmung der Fundamentalfrequenz	17
4.1.1	Autokorrelation	17
4.1.2	YIN-Algorithmus	17
4.1.3	Fourier- und Cepstrum-Analyse	17
4.1.4	Moderne Deep-Learning-Ansätze	17
4.1.5	Auswahl des Verfahrens	18
4.2	Filter	18
4.2.1	Bandpass	18
4.2.2	Moving Average für gestreamte Messdaten	18
4.2.3	Volume Threshold	18
5	Software Entwicklung/Implementierung	18
5.1	Anforderungen	19
5.2	User Journeys	19
5.2.1	Gerhard Gitarrist (Erstnutzer mit Floyd-Rose-Tremolo-Gitarre)	19
5.2.2	Arnold Abbrecher (Wiederholungsnutzer mit Abbruchwunsch)	20
5.2.3	Norman Normaler (Nutzer einer Standard-Gitarre ohne Tremolo)	20
5.3	Anforderungen	20
5.4	Konzeption und Design	20
5.5	Architektur	20
5.6	Implementierung	20
6	Evaluation	21
6.1	Funktionsfähigkeit des Algorithmuses	21
6.2	Erfüllung der Requirements aus SWE	21
6.3	Nutzertests	21
6.3.1	Nutzer 0	21
6.3.2	Nutzer 1	21
7	Ausblick	21
7.1	Usability Verbessern	21
7.2	Indikator was noch nicht überprüft wurde	21
7.3	Mehrere Saiten gleichzeitig Messen	21

7.4 Implementierung für VST und Digital Audio Workstations (Plugin)	21
7.5 Synthetischen Ton zur Kontrolle abspielen um zu hören ob die korrekte Frequenz erfasst wurde.	22
7.6 Erkennen welche Saite gespielt wird, annahme 6 Saiten	22
8 Literatur	22
Bibliografie	25

1 Motivation

Es gibt ein Problem beim Stimmen von Floyd-Rose-Gitarren. Bei diesen Gitarren wird eine Saite zwischen dem Gitarrenkopf und einer bis zu einem gewissen Grad rotierbaren Brücke gespannt. An der Brücke halten unterhalb des Drehpunkts Federn dagegen, wenn man Saiten einspannt. Die Brücke wird gemeinhin als „Floating Bridge“ bezeichnet, weil sie nicht wie herkömmliche Tremolos am Gitarrenkörper aufliegt, sondern zusätzlichen Rotationsspielraum in Richtung des Gitarrenkörpers hat.

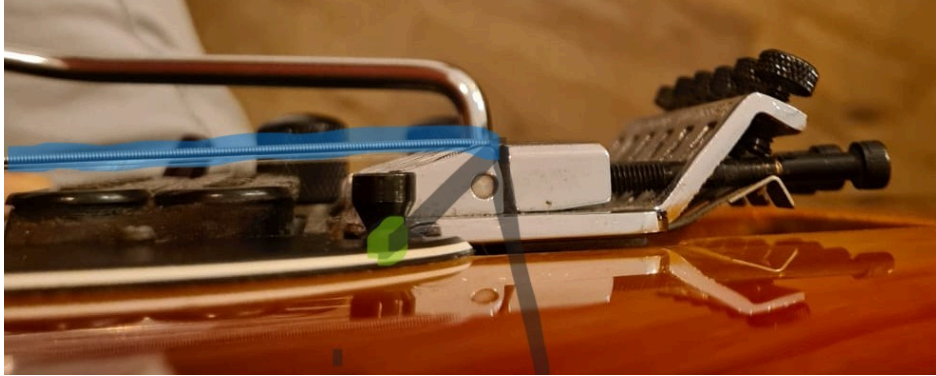


Abbildung 1: Floyd-Rose-Tremolo Bild

Das Floyd-Rose-Tremolo hat einen Hebel, den man ziehen oder drücken kann. Beim Musizieren ändert das den Ton. Auch wenn diese Architektur beliebt ist, weil sie neue Klänge ermöglicht, erschwert sie das Stimmen der Gitarre erheblich. Beim Stimmen erhöht oder verringert man die Spannung einer Saite. Aber das führt dazu, dass die anderen Saiten verstimmt werden. Es gibt Erfahrungsberichte und Aufzeichnungen, wie man eine solche Gitarre effizient stimmen kann. Dabei dauerte das Stimmen 8 Minuten [1]. In Foren sprachen Nutzer von einer Stimmzeit von bis zu 20-30 Minuten, je nachdem, wie sauber und wie viele Saiten sie stimmen mussten [2]. Ziel der Arbeit ist eine App zu entwickeln, die diesen Stimmvorgang beschleunigt.

2 Grundlagen (Physik der Gitarre)

Die Gitarre spannt 6 Saiten zwischen Brücke und Sattel. Die Saiten schwingen in einer bestimmten Frequenz. Beim Stimmen wickelt man die Saite um den Stimmwirbel, sodass sich Spannung und Frequenz ändert. Dass Saiten elastisch sind, wird im folgenden Experiment gezeigt:



Abbildung 2: Gitarre Stimmwirbel

2.1 Experiment: Elastizität von Gitarrensaiten

Die Ergebnisse des Experiments stammen aus der Arbeit des Autors, welche im Rahmen des Moduls „Projekt 3“ aus dem Telekommunikationsinformatik-Studium an der HTWK-Leipzig [3] angefertigt wurde.

In der folgenden Tabelle sind Bilder, die die elastische Dehnung der Saite zeigen:

Bund	Pacifica - Ruhe	Pacifica - Spannung
1	 Abbildung 3: Bund 1 - Ruhe	 Abbildung 4: Bund 1 - Spannung
2	 Abbildung 5: Bund 2 - Ruhe	 Abbildung 6: Bund 2 - Spannung

Bund	Pacifica - Ruhe	Pacifica - Spannung
4	 Abbildung 7: Bund 4 - Ruhe	 Abbildung 8: Bund 4 - Spannung
6	 Abbildung 9: Bund 6 - Ruhe	 Abbildung 10: Bund 6 - Spannung
8	 Abbildung 11: Bund 8 - Ruhe	 Abbildung 12: Bund 8 - Spannung
12	 Abbildung 13: Bund 12 - Ruhe	 Abbildung 14: Bund 12 - Spannung
16	 Abbildung 15: Bund 16 - Ruhe	 Abbildung 16: Bund 16 - Spannung

Bund	Pacifica - Ruhe	Pacifica - Spannung
22	 Abbildung 17: Bund 22 - Ruhe	 Abbildung 18: Bund 22 - Spannung

Die Markierungen, die sich näher am Sattel befanden, legten eine deutlich größere Strecke zurück als jene in unmittelbarer Nähe der Brücke. Die beobachtete Verschiebung nahm kontinuierlich vom Sattel in Richtung Brücke ab.

Die Kontrollmarkierungen auf den übrigen Saiten zeigten dagegen keine oder lediglich eine kaum wahrnehmbare Bewegung. Dies spricht dafür, dass die beobachtete Verschiebung nicht durch ein Verformen des Instruments verursacht wurde, sondern auf eine tatsächliche Längenänderung der gespannten Saite zurückzuführen ist. Als die Saite wieder entspannt wurde, waren die Markierungen wieder an ihrer Ausgangsposition. Die Schwingungsfrequenz der Saite war auch wieder dieselbe wie zu Beginn.

Die Beobachtungen belegen das elastische Verhalten von Gitarrensaiten. Wird die Spannung durch Aufwickeln am Stimmwirbel erhöht, verschieben sich die aufgeklebten Markierungen entlang der Saite in unterschiedlichem Ausmaß. Markierungen in der Nähe der Brücke, die als nahezu fixer Punkt wirkt, erfahren nur eine sehr geringe Verschiebung, während weiter entfernte Markierungen deutlich stärker wandern.

2.2 Physikalisches Modell

Im Folgenden wird ein physikalisches Modell der Gitarre beschrieben, um zu verstehen warum die Floyd-Rose-Gitarre so schwierig zu stimmen ist.

Die Gitarre wird als Abbildung modelliert, die 6 Aufwickelstrecken $\overrightarrow{\Delta L} = \begin{pmatrix} \Delta L_1 \\ \vdots \\ \Delta L_i \\ \vdots \\ \Delta L_6 \end{pmatrix}$ auf einen Frequenz-

vektor $\vec{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_i \\ \vdots \\ f_6 \end{pmatrix}$ abbildet $\overrightarrow{\Delta L} \rightarrow \vec{f}$, wobei jede Komponente zu einer Saite gehört. Beim Stimmen

muss $\overrightarrow{\Delta L}$ so gewählt werden, dass genau die gewünschten Frequenzen erreicht werden. Das Ziel ist die Funktion $f(\overrightarrow{\Delta L})$ zu bestimmen. Der Zusammenhang zwischen effektiver Saitenlänge $L_{S,i}$, Zugkraft $F_{S,i}$, linearer Massendichte μ_i und Frequenz f_i wird durch das Mersennesche Gesetz beschrieben [4]:

$$f_i = \frac{1}{2L_{S,i}} \sqrt{\frac{F_{S,i}}{\mu_i}} \quad (1)$$

Zunächst wird die Saitenkraft $F_{S,i}$ als Funktion der Aufwickelstrecken $\overrightarrow{\Delta L}$ bestimmt. Die Kraft die auf die Saite wirkt, wird durch das Hooksche Gesetz beschrieben [5]:

$$F_{S,i} = (L_{S,i} - L_{0S,i}) \cdot k_{S,i} \quad (2)$$

$L_{0S,i}$ beschreibt die unbelastete Saitenlänge im Abschnitt zwischen Sattel und Brücke. Diese Länge wird durch die Aufwickelstrecke ΔL_i beeinflusst. $L'_{0S,i}$ sei die initiale unbelastete Saitenlänge.

$$L_{0S,i} = L'_{0S,i} - \Delta L_i \quad (3)$$

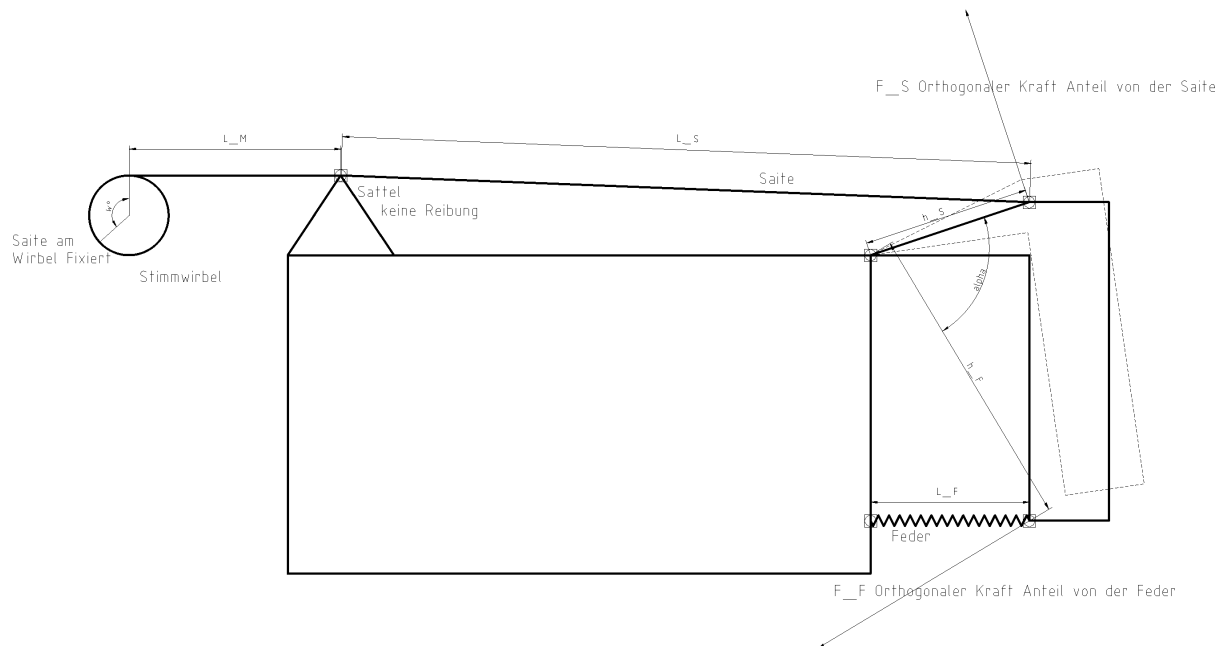


Abbildung 19: Floyd-Rose-Modell Quer

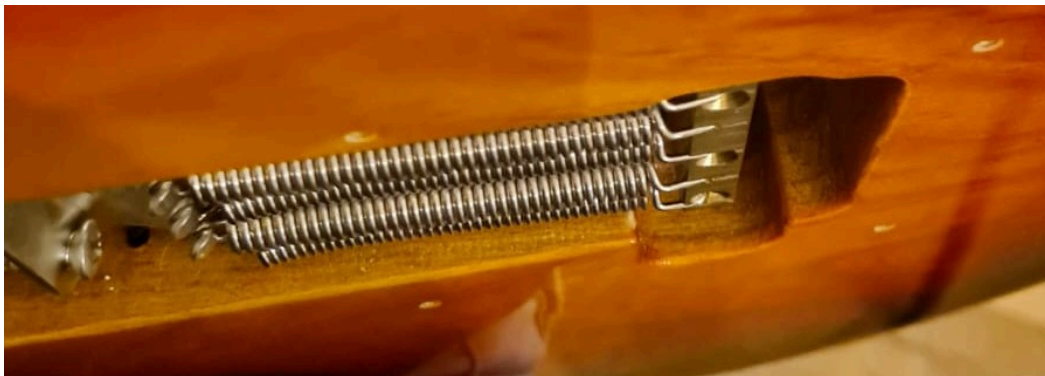


Abbildung 20: Tremolofedern

In Abbildung 1, Abbildung 19 und Abbildung 20 ist zu sehen, wie die Brücke die Tremolofedern und die Saiten über einen Drehmoment koppelt. Die Tremolofedern dienen unterhalb der Brücke als Gegenkraft zu der Saitenspannung.

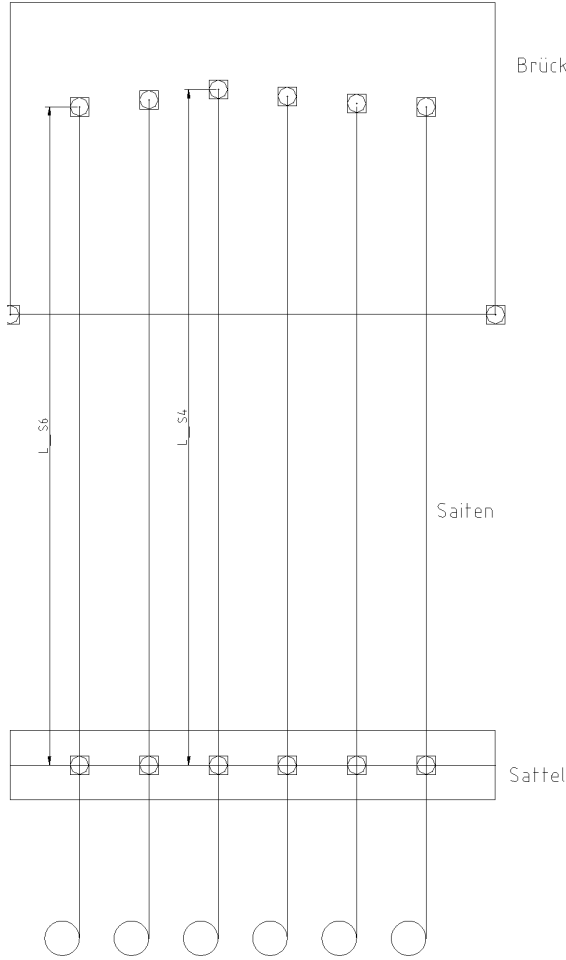


Abbildung 21: Floyd-Rose-Modell Draufsicht



Abbildung 22: Floyd-Rose Draufsicht

In der Realität hat jede Saite ihre eigene Saitenlänge, wie in Abbildung 21 und Abbildung 22 zu sehen ist. Sie variieren zwar nur minimal haben aber einen Einfluss auf die rotatorische Projektion der Kräfte.

Die Brücke wird als starrer, gewinkelter Hebel betrachtet, siehe Abbildung 19. Die Drehachse liege im Koordinatenursprung. Die Vektoren $\vec{h}_{\hat{F}}$ (Hebelarm der Feder) und $\vec{h}_{S,i}$ (Hebelarm der Saite i) schließen konstruktionsbedingt einen konstanten Winkel α_i ein. Die Beträge $h_{\hat{F}}$ und $h_{S,i}$ sind systemspezifische Konstanten. Jede Saite erhält ihren eigenen Hebelarm $\vec{h}_{S,i}$, um den Aufbau wie in Abbildung 22 und Abbildung 21 korrekt zu modellieren. Die Tremolofedern erhalten in diesem Modell einen gemeinsamen Hebelarm $\vec{h}_{\hat{F}}$.

$$\vec{h}_{\hat{F}}(\beta) = h_{\hat{F}} \begin{pmatrix} \cos(\beta) \\ \sin(\beta) \end{pmatrix} \quad (4)$$

$$\vec{h}_{S,i}(\beta) = h_{S,i} \begin{pmatrix} \cos(\beta + \alpha_i) \\ \sin(\beta + \alpha_i) \end{pmatrix} \quad (5)$$

Sei $\vec{P}_S$ die konstante Position des Sattels. Die effektive Saitenlänge und Tremolofederlänge ergeben sich zu

$$L_{S,i}(\beta) = |\vec{h}_{S,i}(\beta) - \vec{P}_S| \quad (6)$$

$$L_{\hat{F}}(\beta) = |\vec{h}_{\hat{F}}(\beta) - \vec{P}_{\hat{F}}| \quad (7)$$

Nun soll die Variable β bestimmt werden, die sich aus dem Kräftegleichgewicht und der darauffolgenden Hebelposition ergibt. Nach den Gesetzen der Statik trägt ausschließlich der zur jeweiligen Hebelarmrichtung orthogonale Kraftanteil zum Drehmoment bei [6]. Im stationären Gleichgewicht gilt das Drehmomentgleichgewicht:

$$\sum_{i=1}^6 F_{S,i,\perp h_{S,i}} \cdot h_{S,i} = F_{\hat{F},\perp h_{\hat{F}}} \cdot h_{\hat{F}} \quad (8)$$

Dabei bezeichnen $F_{S,i,\perp h_{S,i}}$ und $F_{\hat{F},\perp h_{\hat{F}}}$ jeweils die Anteile der Kräfte $\overrightarrow{F_{S,i}}$ und $\overrightarrow{F_{\hat{F}}}$, die orthogonal zu den Hebelarmen $h_{S,i}$ und $h_{\hat{F}}$ wirken. Auf der linken Seite von Gleichung 8 müssen die Kräfte der 6 Saiten aufaddiert werden, da sich die Kräfte parallelgeschalteter Federn addieren [7].

Zunächst wird der Richtungsvektor von $F_{S,i}$, $h_{S,i}$, $F_{\hat{F}}$ und $h_{\hat{F}}$ normiert, wobei $P_{\hat{F}}$ der Punkt ist, an dem die Tremolofeder an der Gitarre befestigt ist.

$$\overrightarrow{e_{F_{S,i}}} = \frac{\overrightarrow{P_S} - \overrightarrow{h_{S,i}}}{|\overrightarrow{P_S} - \overrightarrow{h_{S,i}}|} \quad (9)$$

$$\overrightarrow{e_{h_{S,i}}} = \begin{pmatrix} \cos(\beta + \alpha_i) \\ \sin(\beta + \alpha_i) \end{pmatrix} \quad (10)$$

$$\overrightarrow{e_{F_{\hat{F}}}} = \frac{\overrightarrow{P_{\hat{F}}} - \overrightarrow{h_{\hat{F}}}}{|\overrightarrow{P_{\hat{F}}} - \overrightarrow{h_{\hat{F}}}|} \quad (11)$$

$$\overrightarrow{e_{h_{\hat{F}}}} = \begin{pmatrix} \cos(\beta) \\ \sin(\beta) \end{pmatrix} \quad (12)$$

Aus der orthogonalen Projektion eines Vektors $\vec{a}$ bezüglich eines Vektors $\vec{b}$ folgt [8]:

$$F_{S,i,\perp h_{S,i}} = F_{S,i} \cdot \sqrt{1 - (\overrightarrow{e_{F_{S,i}}} \cdot \overrightarrow{e_{h_{S,i}}})^2} \quad (13)$$

$$= F_{S,i} \cdot \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \quad (14)$$

Analog ergibt sich für die Tremolofeder:

$$F_{\hat{F},\perp h_{\hat{F}}} = F_{\hat{F}} \cdot \sqrt{1 - (\overrightarrow{e_{F_{\hat{F}}}} \cdot \overrightarrow{e_{h_{\hat{F}}}})^2} \quad (15)$$

$$= F_{\hat{F}} \cdot \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \quad (16)$$

Das Kräftegleichgewicht lässt sich damit schreiben als:

$$\sum_{i=1}^6 F_{S,i} \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \cdot h_{S,i} = F_{\hat{F}} \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \cdot h_{\hat{F}} \quad (17)$$

Der nächste Schritt wäre, diesen Ausdruck nach $\beta(\overline{\Delta L})$ umzustellen, um die Hebelposition zu bestimmen. Allerdings ist dies nicht analytisch möglich, da β in den Sinusfunktionen und den Hebelarmvektoren auf beiden Seiten der Gleichung vorkommt. Es liegt ein nichtlineares Gleichungssystem vor, das numerisch gelöst werden muss.

Bringt man sie in die Form einer Nullstellengleichung, erhält man

$$0 = g(\beta; \overrightarrow{\Delta L}) = \sum_{i=1}^6 F_{S,i} \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \cdot h_{S,i} - F_{\hat{F}} \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \cdot h_{\hat{F}} \quad (18)$$

Damit liegt ein eindimensionales nichtlineares Optimierungs- bzw. Nullstellenproblem vor, mit dem sich der Rotationswinkel β numerisch bestimmen lässt.

Aus dem so berechneten Winkel ergeben sich transitiv die abhängigen Größen $h_{S,i}(\beta)$, $L_{S,i}(\beta)$ und $F_{S,i}(\overrightarrow{\Delta L})$.

μ_i ist von der Aufwickelstrecke $\overrightarrow{\Delta L}$ abhängig. Im Allgemeinen gilt für Saite i :

$$\mu_i = \frac{m_i}{L_{S,i,\text{Total}}} \quad (19)$$

Dabei bezeichnet m_i die Gesamtmasse und $L_{S,i,\text{Total}}$ die Gesamtlänge der Saite i . Die Gesamtlänge setzt sich zusammen aus der effektiven Saitenlänge $L_{S,i}$ und der Teil der Saite der hinter dem Sattel liegt, wie in Abbildung 2 und Abbildung 21 zu sehen ist.

Diese Strecke sei $L_{M,i} = L_{0M,i} + \Delta L_i$.

$$L_{S,i,\text{Total}}(\overrightarrow{\Delta L}) = L_{S,i}(\overrightarrow{\Delta L}) + L_{0M,i} + \Delta L_i \quad (20)$$

Beim Aufwickeln der Saite erhöht sich die Strecke hinter dem Sattel um ΔL_i . Die zusätzliche Strecke, die durch die Dehnung entsteht steckt in $\Delta L_{S,i}(\overrightarrow{\Delta L})$.

Die lineare Massendichte ergibt sich somit zu:

$$\mu_i(\overrightarrow{\Delta L}) = \frac{m_i}{L_{S,i}(\overrightarrow{\Delta L}) + L_{0M,i} + \Delta L_i} \quad (21)$$

Darauf aufbauend lässt sich eine Abbildung definieren, die die Aufwickelstrecken jeder Saite auf einen Frequenzvektor abbildet:

$$f_i(\overrightarrow{\Delta L}) = \frac{1}{2 \cdot L_{S,i}(\overrightarrow{\Delta L})} \sqrt{\frac{F_{S,i}(\overrightarrow{\Delta L})}{\mu_i(\overrightarrow{\Delta L})}} \quad (22)$$

Es wird ersichtlich, dass die Aufwickelstrecken der Saiten die Frequenzen aller Saiten beeinflussen. Das erklärt, warum das Stimmen einer Floyd-Rose-Gitarre so schwierig ist.

Beim Stimmen werden die Aufwickelstrecken nur in kleinen Schritten verändert. In diesem Fall verhält sich das System näherungsweise linear. Da das System physikalisch ist, können wir das System als stetig betrachten.

2.3 Experiment: Nachweis Linearität

Diese Linearität wurde bereits experimentell in der *Projekt 3* Arbeit des Autors [3] überprüft.

2.3.1 Vorgehensweise

Zunächst wurde jede Saite in eine Ausgangsposition gebracht. Die Ausgangsfrequenzen der Saiten wurden in Hertz gemessen. Anschließend wurde jeweils eine Saite um ein beliebiges Δ (in Hertz) verstimmt. Dieses Δ wurde so gewählt, dass die Verstimmung deutlich hörbar ist. Für jeden Schritt wurde die Frequenz aller anderen Saiten gemessen.

Nr.	Saite	Frequenz
1	E2 = E-Saite	82.41 Hz
2	A2 = A-Saite	110 Hz
3	D3 = D-Saite	146.83 Hz
4	G3 = G-Saite	196 Hz
5	B3 = B-Saite	246.94 Hz
6	E4 = hohe E-Saite	329.63 Hz

2.3.2 Ergebnisse

2.3.2.1 Relative Visualisierung der Frequenzänderungen

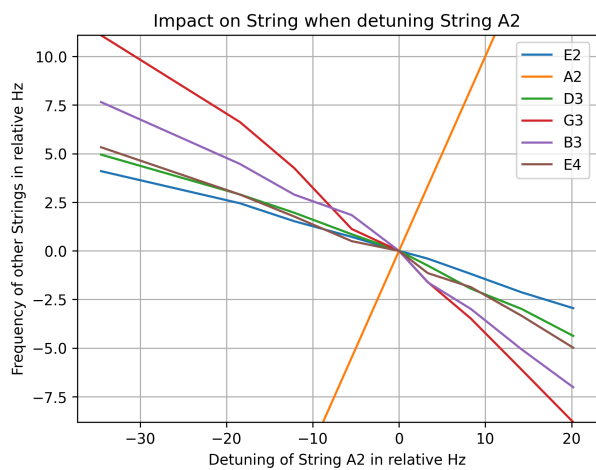
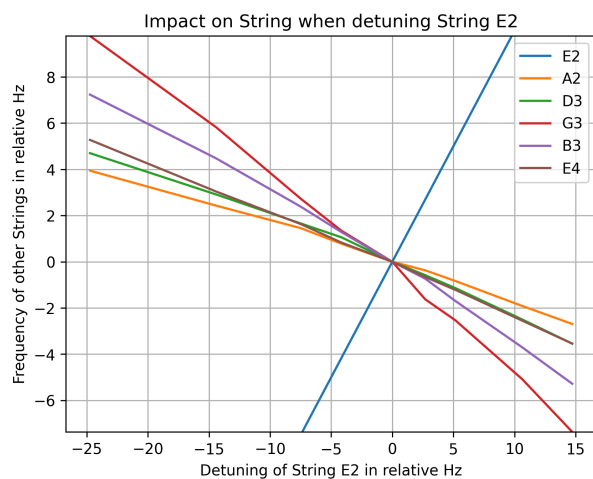


Abbildung 23: Relativer Einfluss der E2 Saite auf die anderen Saiten

Abbildung 24: Relativer Einfluss der A2 Saite auf die anderen Saiten

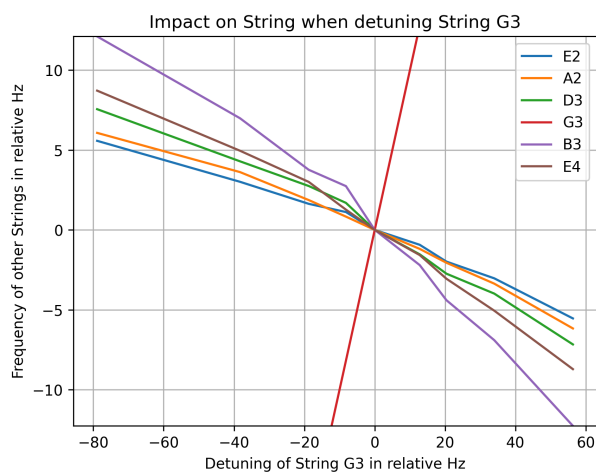
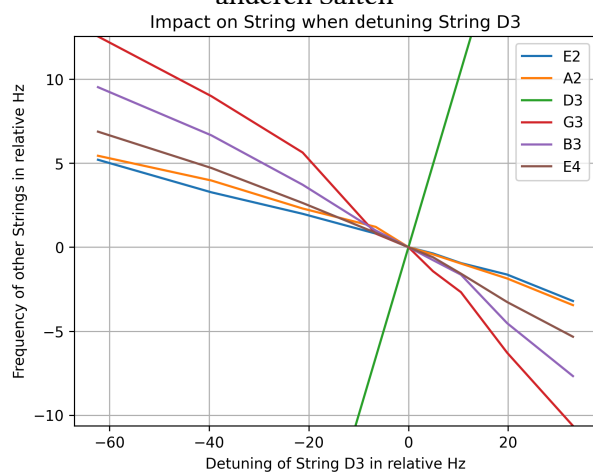


Abbildung 25: Relativer Einfluss der D3 Saite auf die anderen Saiten

Abbildung 26: Relativer Einfluss der G3 Saite auf die anderen Saiten

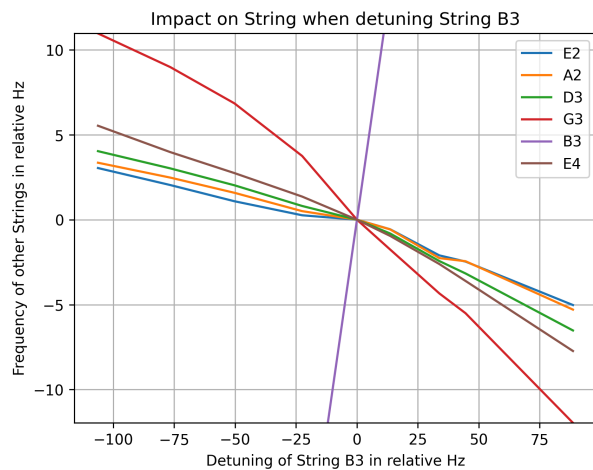


Abbildung 27: Relativer Einfluss der B3 Saite auf die anderen Saiten

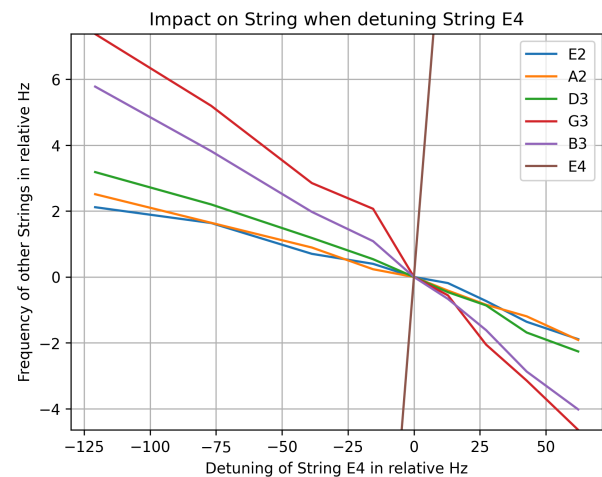


Abbildung 28: Relativer Einfluss der E4 Saite auf die anderen Saiten

2.3.2.2 Korrelationskoeffizienten und Fehler

Pearson Correlation

Y values (correlation input)

X values (correlation input)

	E2	A2	D3	G3	B3	E4
E2	1.0	-0.997	-0.997	-0.991	-0.98	-0.982
A2	-0.998	1.0	-0.994	-0.991	-0.983	-0.995
D3	-0.996	-0.993	1.0	-0.99	-0.982	-0.993
G3	-0.998	-0.996	-0.988	1.0	-0.993	-0.992
B3	-0.998	-0.989	-0.99	-0.988	1.0	-0.994
E4	-0.999	-0.993	-0.993	-0.991	-0.987	1.0

Abbildung 29: Pearson-Korrelationskoeffizienten zwischen gemessenen und erwarteten Frequenzänderungen

Während der Durchführung des Experiments fiel auf, dass beim Zurückbringen einer Saite in ihre Ausgangsposition alle anderen Saiten ebenfalls wieder ihre ursprüngliche Frequenz annahmen.

2.3.3 Diskussion der Ergebnisse

Das System ist elastisch, da Ausgangs- und Endfrequenzen nach jedem Durchgang gleich sind.

Die Linearität des Systems ist nicht perfekt, aber hinreichend gut für kleine Verstimmungen. Sie lässt sich quantitativ mit dem Korrelationskoeffizienten nach Bravais-Pearson [9] zwischen gemessenen und erwarteten Frequenzänderungen jeder Saite bestimmen. In Abbildung 29 sind die Korrelationskoeffizienten für jede Saite dargestellt. Der Betrag aller Werte liegt über 0.98, was auf eine sehr starke positive Korrelation hinweist. Das rechtfertigt die Annahme einer linearen Beziehung für kleine Änderungen.

2.4 Mathematische Lösung

Die Frequenzen der Saiten können als Vektor dargestellt werden:

$$\vec{f}_0 = \begin{pmatrix} f_{E2} \\ f_{A2} \\ f_{D3} \\ f_{G3} \\ f_{B3} \\ f_{E4} \end{pmatrix} \quad (23)$$

$$C = \begin{pmatrix} 1 & c_{12} & c_{13} & c_{14} & c_{15} & c_{16} \\ c_{21} & 1 & c_{23} & c_{24} & c_{25} & c_{26} \\ c_{31} & c_{32} & 1 & c_{34} & c_{35} & c_{36} \\ c_{41} & c_{42} & c_{43} & 1 & c_{45} & c_{46} \\ c_{51} & c_{52} & c_{53} & c_{54} & 1 & c_{56} \\ c_{61} & c_{62} & c_{63} & c_{64} & c_{65} & 1 \end{pmatrix}, \vec{g} = \begin{pmatrix} \hat{f}_{E2} \\ \hat{f}_{A2} \\ \hat{f}_{D3} \\ \hat{f}_{G3} \\ \hat{f}_{B3} \\ \hat{f}_{E4} \end{pmatrix} \quad (24)$$

- $\vec{f}_0$: Ausgangsfrequenzen der Saiten, gemessen z.B. mit einem digitalen Stimmgerät
- C : Verstimmungsmatrix, wobei c_{ij} den Verstimmungsfaktor der Saite i angibt, wenn die Saite j um 1Hz verstimmt wird
- $\vec{g}$: Ziel-Frequenzen nach der Verstimmung

Die Verstimmungsmatrix aus dem Experiment ist in Abbildung 30 dargestellt:

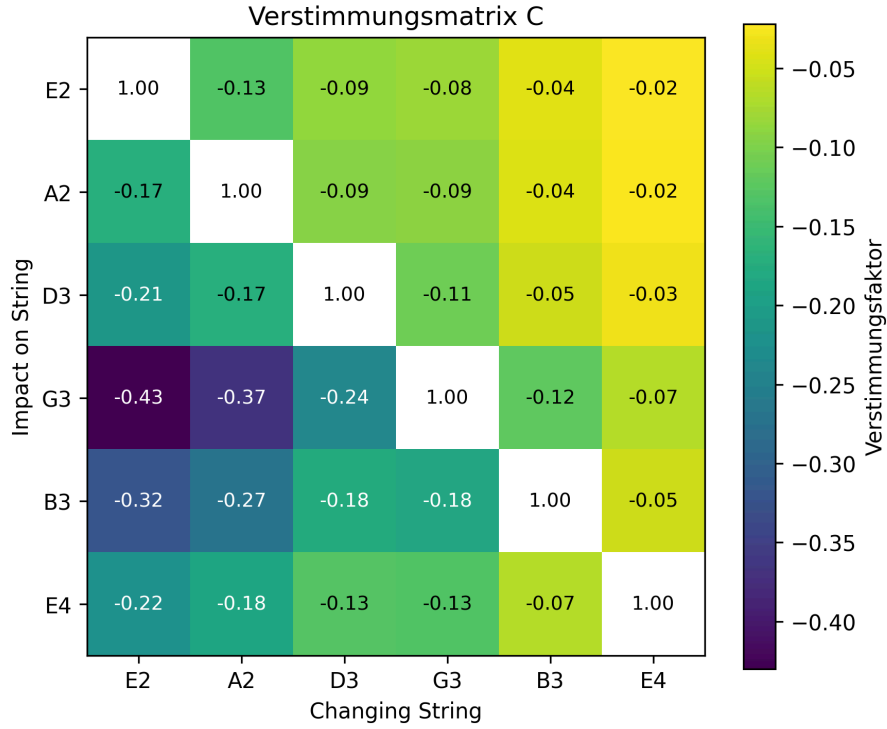


Abbildung 30: Verstimmungsmatrix Beispiel C

Der Vektor

$$\vec{\Delta} = \begin{pmatrix} \Delta_{E2} \\ \Delta_{A2} \\ \Delta_{D3} \\ \Delta_{G3} \\ \Delta_{B3} \\ \Delta_{E4} \end{pmatrix} \quad (25)$$

gibt an, um wie viel jede Saite verstimmt werden muss.

Die effektive Verstimmung wird durch die Multiplikation mit der Verstimmungsmatrix berechnet:

$$C \cdot \vec{\Delta} = \vec{\Delta}_{\text{effective}} \quad (26)$$

Damit die Ziel-Frequenzen $\vec{g}$ erreicht werden, gilt:

$$\vec{g} = \vec{f}_0 + \vec{\Delta}_{\text{effective}} \rightarrow \vec{\Delta}_{\text{effective}} = \vec{g} - \vec{f}_0 \quad (27)$$

Um die Eingangsverstimmung $\vec{\Delta}$ zu bestimmen, muss das Inverse der Matrix C gebildet werden:

$$C \cdot \vec{\Delta} = \vec{\Delta}_{\text{effective}} \rightarrow \vec{\Delta} = C^{-1} \cdot \vec{\Delta}_{\text{effective}} \quad (28)$$

$$\vec{\Delta} = C^{-1} \cdot (\vec{g} - \vec{f}_0) \quad (29)$$

C^{-1} ist die Inverse der Verstimmungsmatrix.

Somit benötigt man für die Berechnung:

1. Ausgangsfrequenzen $\vec{f}_0$
2. Ziel-Frequenzen $\vec{g}$
3. Verstimmungsmatrix C

3 Ablauf eines Stimmvorgangs

Der Ablauf eines Stimmvorgangs beschreibt welche Schritte durchlaufen müssen um letztendlich die Gitarre zustimmen. Im folgenden wird der Schritte beschrieben und mit welchem Konkreten verfahren der Schritt umgesetzt werden soll.

3.1 Initierung - Bestimmung der Verstimmungsmatrix

Um nun eine Gitarre zu stimmen, muss zunächst die Verstimmungsmatrix, der zu stimmenden Gitarre ermittelt werden. Das muss für eine Gitarre nur einmal gemacht werden.

Für jeden Eintrag der Matrix benötigt man mindestens 2 Messpunkte um die Steigung zu bestimmen. Die Ausnahme sind die Diagonalen Elemente, da die Steigung dort immer 1 ist.

Der eine Messpunkt beschreibt die Ausgangslage der Gitarre und die Folgenden das Verhalten wenn sich eine Saite verändert. Um Die Präzision zu erhöhen kann eine quasi beliebige Anzahl von Messpunkten aufgenommen werden. Um daraus die korrekte Steigung zu bestimmen benötigen wir ein Lineares Regressionsverfahren.

Dabei kommen folgende Verfahren in Frage:

Kleinste Quadrate (OLS) Minimiert die Summe der quadrierten vertikalen Abstände in Y -Richtung zwischen den Datenpunkten und der Regressionsgeraden. Sie wird verwendet, wenn nur die Antwortvariable Y fehlerbehaftet ist und die Prädiktorvariable X als exakt betrachtet wird.

Orthogonale Regression (Deming-Regression) Minimiert die Summe der quadrierten senkrechten (orthogonalen) Abstände der Punkte von der Geraden. Sie wird angewendet, wenn sowohl die Antwortvariable Y als auch die Prädiktorvariable X Messfehler enthalten. [10]

Da auch die X Werte Fehler in der Messung enthalten, wird die **Orthogonale Regression** verwendet.

3.2 Stimmen der Gitarre

Man definiert, welche Zielfrequenz die Gitarre haben soll. Es gibt unterschiedliche *Stimmungen* bei Gitarren, da sie manche *Akkorde* vereinfachen oder erst möglich machen.

Dann misst man einmalig den Zustand der Gitarre. Um den Frequenzvektor zu erhalten. Mit einer Implementierung von Gleichung 29 wird dann Bestimmt um wieviel Herz welche Saite verstimmt werden muss.

3.3 Überprüfung

Zum Schluss wird überprüft, ob die Gitarre gestimmt wurde, mit einem Herkömmlichen Stimmgerät.

4 Grundlegende Verfahren

Es gibt verfahren die für die Umsetzung notwendig sind.

4.1 Bestimmung der Fundamentalfrequenz

Für die Implementierung eines Tuners auf mobilen Geräten ist die präzise und effiziente Bestimmung der Grundfrequenz (Fundamental Frequency, F0) entscheidend. Im Folgenden werden gängige Verfahren vorgestellt und hinsichtlich Genauigkeit, Rechenaufwand und Anwendungsbereich bewertet.

4.1.1 Autokorrelation

Die Idee der Autokorrelation besteht darin, dass ein Signal mit einer Periode von τ , wenn es mit sich selbst gefaltet wird, bei vielfachen von τ Maxima aufweisen wird. Wenn man nun den ersten von null verschiedenen x Wert wählt, der bei einem Hochpunkt ist, hat man mit hoher Wahrscheinlichkeit, das Richtige T und mit seinem Inversen die Frequenz des Signals.

$$r_{t(\tau)} = \sum_{j=t+1}^{t+W} x_j x_{j+\tau} \quad (30)$$

wobei $r_{t(\tau)}$ die Autokorrelationsfunktion von der Verzögerung τ berechnet zum Zeitindex t und W ist die Integrationsfenstergröße.[11]

4.1.2 YIN-Algorithmus

Der YIN-Algorithmus ist eine Weiterentwicklung der Autokorrelation. Sie fügt extra Fehlerreduktionsschritte hinzu. So wird zum Beispiel durch die „Difference function“ eine Immunität gegenüber schwankenden Amplituden erzeugt und uninteressante τ werden herausgefiltert.[11] Der Algorithmus wird in vielen Stimmgeräten verwendet.

4.1.3 Fourier- und Cepstrum-Analyse

Bei der Fourier-Analyse wird das Signal ins Frequenzspektrum transformiert, um das Spektrum nach der Grundfrequenz zu durchsuchen. Die Cepstrum-Analyse erweitert diesen Ansatz, indem das logarithmierte Spektrum erneut transformiert wird, um periodische Muster zu detektieren. Eine Analyse hat gezeigt, dass jedoch Fourier-Analysen fehleranfällig sind und eine hohe Sampling-Rate benötigen.[12] Ein Vorteil von Cepstrum ist die Robustheit gegenüber harmonischen Obertönen und eine gute Integration in digitale Signalverarbeitungssysteme. Ein Nachteil ist die eingeschränkte Genauigkeit bei niedrigen Frequenzen oder verrauschten Signalen, da die Cepstrum-Analyse auf der Annahme basiert, dass das Signal periodisch ist und dass die Obertöne harmonisch sind. In realen Situationen können diese Annahmen jedoch nicht immer erfüllt sein, was zu Fehlern bei der Schätzung der Grundfrequenz führen kann. [13]

4.1.4 Moderne Deep-Learning-Ansätze

Neuronale Netze wie CREPE oder DeepPitch nutzen Convolutional- oder Recurrent Neural Networks, um direkt aus Roh-Audio oder Spektrogrammen die Fundamental-Frequenz vorherzusagen.

Vorteile sind die hohe Robustheit bei Polyphonie, Hintergrundgeräuschen und unterschiedlichen Instrumenten.

Nachteile sind der hohe Rechenaufwand, die Notwendigkeit großer Trainingsdatensätze und die Ressourcenintensität auf Mobilgeräten. [14]

4.1.5 Auswahl des Verfahrens

Für die Implementierung wird der YIN-Algorithmus gewählt, da er eine gute Balance zwischen Genauigkeit und Rechenaufwand bietet. Er ist speziell für die Schätzung der Grundfrequenz entwickelt worden und bietet eine robuste Leistung bei verschiedenen Signalbedingungen. Hinzu kommt, dass er bereits in vielen Stimmgeräten erfolgreich eingesetzt wird und bereits Implementierungen in verschiedenen Programmiersprachen verfügbar sind.

4.2 Filter

Damit die Fundamental Frequenz besser erfasst werden kann, ist es sinnvoll, das Signal vorher zu bereinigen und Störgeräusche zu filtern.

4.2.1 Bandpass

Da der YIN-Algorithmus verwendet wird, kann man durch die Anpassung seiner Parameter, den Effekt eines Bandpasses erzielen. Verringert man die Samplingrate, so werden höhere Frequenzen nicht mehr erkannt. Das liegt an dem *Nyquist-Shannon sampling theorem*[15]. Um die Tief Frequenz zu filtern, kann man nun die Samplelänge beeinflussen. Mit $f = \frac{1}{T}$ darf die Samplelänge nicht länger sein als $T_{\max} = \frac{1}{f_{\text{lowest}}}$. Der Frequenzbereich einer Gitarre liegt ungefähr zwischen 50 Hz – 350 Hz

4.2.2 Moving Average für gestreamte Messdaten

Mit einem Mikrofon werden auch Geräusche der Umwelt aufgenommen. Das führt zu Schwankungen in der Frequenz Messung. Damit die Visualisierung dieser Messdaten nicht zu sehr ruckeln und *smoother* werden kann man den Average, der letzten N Messdaten nehmen und diesen anzeigen.

4.2.3 Volume Threshold

Damit die Fundamentalfrequenz nicht ständig ermittelt wird, ist es sinnvoll diese nur zu aktivieren, wenn das Signal eine gewisse Mindest Lautstärke erreicht.

$$\text{dB} = 20 \cdot \frac{\log\left(\frac{\text{rms}}{32768}\right)}{\ln(10)}, \quad (31)$$

5 Software Entwicklung/Implementierung

Die Software soll nach dem Buch „Mobile App Engineering“ [16] entwickelt werden.

Das Buch beschäftigt sich mit der Entwicklung von *Enterprise Apps*. Die in diesem Rahmen entwickelte App ist zwar keine *Enterprise App*, aber die Prinzipien der Softwareentwicklung, die in diesem Buch beschrieben werden, sind dennoch anwendbar. Es werden insbesondere die Prinzipien der Anforderungsanalyse und der nutzerzentrierten Gestaltung.

Damit die App frühzeitig auf sovielen Geräten wie möglich verwendet werden kann, wird die App mit einem Cross-Plattform-Framework entwickelt. Es gibt verschiedene Frameworks, die für die Entwicklung von Cross-Plattform-Apps verwendet werden können, wie zum Beispiel React Native [17], .NET MAUI (ehemals Xamarin)[18] und Flutter [19].

Aufgrund von eigener Erfahrungen mit React Native, Dot Net Maui (ehemals Xamarin) und Flutter wurde Flutter ausgewählt. Flutter bot eine bessere Performance als React Native, und das Designen

von Benutzeroberflächen war mit Flutter einfacher als mit Dot Net Maui. Die Developer Experience ist bei Flutter besonders gut.

Somit wird das *Cross-Compiling* Paradigma angewandt. Die App benötigt Zugriff auf das Mikrofon des Endgeräts und wird in Echtzeit Frequenzanalysen und weitere mathematische Operationen durchführen. Mit Flutter sieht die App auf jedem Gerät gleich aus. In dem Buch „Mobile App Engineering“ (Jahr 2017) [16] wurden Nachteile von *Cross-Compiling* aufgezählt die heute von Flutter gelöst werden.

5.1 Anforderungen

Im Rahmen der Anforderungsanalyse und der nutzerzentrierten Gestaltung der mobilen Tuning-App wurden drei repräsentative User Journeys erarbeitet. Diese beschreiben typische Nutzungsszenarien unterschiedlicher Nutzergruppen und dienen der Validierung der funktionalen sowie der interaktionsbezogenen Anforderungen. Die Journeys wurden auf Basis der erstellten Personas und der identifizierten Pain Points entwickelt und berücksichtigen sowohl Erstnutzung als auch wiederkehrende Nutzungsszenarien.

5.2 User Journeys

5.2.1 Gerhard Gitarrist (Erstnutzer mit Floyd-Rose-Tremolo-Gitarre)

Gerhard Gitarrist besitzt eine Floyd-Rose-Tremolo-Gitarre und möchte diese präzise stimmen. Er hat von der App erfahren und lädt sie aus dem App Store herunter. Beim erstmaligen Start der App gelangt er auf eine Onboarding-Seite, die ihm bewusst macht, dass er mit der Funktionsweise der App noch nicht vertraut ist.

Auf dieser Startseite werden ihm zwei Hilfsangebote prominent angezeigt: ein Link zu einem kurzen Erklärvideo (ca. 90 Sekunden) sowie ein prägnanter, textbasierter Erklärtext. Gerhard entscheidet sich jedoch gegen beide Hilfsmittel, da er sofort mit dem Stimmvorgang beginnen möchte. Er klickt daher auf den prominent platzierten Button „Direkt zum Stimmgerät“, der ihn ohne weitere Verzögerung auf die Tuning-Seite führt. Die App informiert ihn nun klar und strukturiert darüber, dass zunächst eine einmalige Konfiguration des Stimmgeräts für seine Floyd-Rose-Gitarre notwendig ist. Der Konfigurationsprozess beginnt automatisch: Die App fordert Gerhard auf, nacheinander alle sechs Saiten einmal anzupspielen, um die aktuelle Grundstimmung und die Saitenreihenfolge zu erfassen. Anschließend wird er aufgefordert, die tiefe E-Saite bewusst um einen definierten Betrag zu verstimmen (z. B. um eine Terz). Sobald die gewünschte Verstimmung erreicht ist, erhält er ein positives visuelles und akustisches Feedback. Danach muss er jede weitere Saite einmal einzeln anspielen. Spielt er versehentlich eine falsche Saite an, kann er über einen deutlich sichtbaren Button „Vorherige Saite“ zum vorherigen Schritt zurückkehren.

Der Vorgang wiederholt sich analog für alle Saiten. Nach Abschluss der Kalibrierung erscheint ein Dialog, der Gerhard fragt, ob er diese Konfiguration speichern möchte, um den Prozess künftig zu überspringen. Neben einem Textfeld zur freien Benennung der Konfiguration (z. B. „Meine Floyd Rose – Standard“) stehen die beiden Buttons „Speichern“ und „Abbrechen“ zur Verfügung. Nach Bestätigung öffnet sich das eigentliche Stimmgerät. Oben in der Kopfzeile wird die gerade ausgewählte Konfiguration angezeigt. Gerhard wird nun aufgefordert, jede Saite einzeln auf die exakte Zieltonhöhe (in Hz) zu stimmen. Sobald alle Saiten korrekt gestimmt sind, erhält er eine abschließende Bestätigung „Gitarre erfolgreich gestimmt“ und kann die App beenden.

5.2.2 Arnold Abbrecher (Wiederholungsnutzer mit Abbruchwunsch)

Arnold Abbrecher ist bereits Nutzer der App und besitzt ebenfalls eine Floyd-Rose-Tremolo-Gitarre. Beim Start der App gelangt er direkt in das Stimmgerät, das automatisch die zuletzt verwendete Konfiguration lädt. Bevor er mit dem eigentlichen Stimmen beginnt, bemerkt er einen Hinweis-Text, der ihn darauf aufmerksam macht, dass bei neuen Saiten oder bei einer anderen Gitarre eine erneute Konfiguration des Stimmgeräts erforderlich ist. Arnold entscheidet sich dafür, eine neue Konfiguration anzulegen, und klickt auf den Button „Neue Konfiguration erstellen“. Der Konfigurationsprozess startet wie bei Gerhard Gitarrist. Nach einigen Schritten wird Arnold jedoch durch äußere Einflüsse (z. B. ein Telefonat oder Zeitmangel) unterbrochen. Er möchte den Vorgang an dieser Stelle abbrechen und klickt auf den jederzeit sichtbaren und gut erreichbaren Button „Konfiguration abbrechen“. Die App fragt sicherheitshalber noch einmal nach einer Bestätigung („Möchten Sie den Konfigurationsprozess wirklich abbrechen? Die bisherigen Daten gehen verloren.“). Nach Bestätigung wird Arnold zurück zur Startseite des Stimmgeräts geleitet; die bisherigen Konfigurationsdaten werden verworfen und die zuletzt gespeicherte Konfiguration bleibt erhalten.

5.2.3 Norman Normaler (Nutzer einer Standard-Gitarre ohne Tremolo)

Norman Normaler ist Gitarrist mit einer herkömmlichen E-Gitarre ohne Floyd-Rose-Tremolo-System. Er besitzt die App bereits und startet sie. Wie bei Arnold öffnet sich direkt das Stimmgerät mit der zuletzt verwendeten Konfiguration. Da Norman keine Floyd-Rose-Gitarre stimmen möchte, sondern lediglich eine schnelle Standardstimmung benötigt, navigiert er über die untere Navigationsleiste zum Menüpunkt „Standard Tuner“. Die App wechselt nahtlos in den vereinfachten Stimmmodus, der auf die klassische chromatische Stimmfunktion ohne spezielle Floyd-Rose-Kalibrierung zurückgreift. Norman spielt nun nacheinander die Saiten an und stimmt sie manuell auf die Standardtonhöhen (E-A-D-G-H-e). Die App zeigt dabei in Echtzeit die aktuelle Frequenz sowie eine visuelle Hilfestellung (Zeiger bzw. Farbverlauf). Sobald alle Saiten korrekt gestimmt sind, erhält er eine Erfolgsmeldung. Norman kann die App anschließend direkt schließen, ohne weitere Konfigurationsschritte durchlaufen zu müssen.

5.3 Anforderungen

Aus User Journeys ableiten

5.4 Konzeption und Design

Screenshots von der App einfügen das hatte zu viele buttons, buttons werden reduziert und usability verbessert, allerdings sind jetzt die Schritte nicht mehr offensichtlich

5.5 Architektur

Backend der App konzipieren. Pipeline Modelle etc.

5.6 Implementierung

Auswahl von Frameworks und Libraries:

- Flutter
- Riverpod
- etc..

6 Evaluation

6.1 Funktionsfähigkeit des Algorithmuses

Mit der App konnte die Gitarre erfolgreich gestimmt werden.

6.2 Erfüllung der Requirements aus SWE

6.3 Nutzertests

6.3.1 Nutzer 0

Die App wurde in einem ruhigen Zimmer, mit verstärkter Gitarre ohne Verzerrungseffekt getestet. Hierbei wurden die Frequenzen der Saiten korrekt erkannt und die Gitarre konnte erfolgreich gestimmt werden. Es gab kleine Schwierigkeiten bei dem Erkennen der Fundamental frequenz. Da diese etwas schwankten.

Innerhalb von ungefähr 7 Minuten war die Gitarre gestimmt.

6.3.2 Nutzer 1

Hierbei wurde die App auf einer Jam Session vorgestellt. Beim Versuch, die Frequenz der E Gitarre zu messen, wurde nicht die korrekte Fundamentalfrequenz erkannt, sondern der Oberton mit Faktor 2. Gerade weil es in der Umgebung laut war und die Gitarre einen Verzerrungseffekt hatte, bei dem Obertöne verstärkt werden. Der Stimmvorgang wurde abgebrochen.

Für so ein Szenario muss die Erkennung der Fundamentalfrequenz stabiler sein.

7 Ausblick

7.1 Usability Verbessern

Die App nutzt noch zu komplizierte Begriffe für die Nutzer.

7.2 Indikator was noch nicht überprüft wurde

Ein Indikator könnte sein, dass die App anzeigt, welche Saiten bereits überprüft wurden.

7.3 Mehrere Saiten gleichzeitig Messen

Fourier Transformation, 6 Peaks erkennen und die obertöne rausrechnen.

7.4 Implementierung für VST und Digital Audio Workstations (Plugin)

VST und CLAP Plugin implementieren, in C++

7.5 Synthetischen Ton zur Kontrolle abspielen um zu hören ob die korrekte Frequenz erfasst wurde.

7.6 Erkennen welche Saite gespielt wird, annahme 6 Saiten

Wenn der Nutzer die Gitarre stimmt, dann wollen wir erkennen welche Saite er verändert und automatisch switchen. Annahme ist, dass er bereits ungefähr an der richtigen Frequenz ist.

8 Literatur

Buch- Mobile App Engineering

Abkürzungsverzeichnis

Sattel Ein fester Punkt am Gitarrenhals (vgl. Abbildung 31).

Brücke Ein fester Punkt am Gitarrenkörper, an dem die Saiten befestigt sind (vgl. Abbildung 31).

Saite Ein Dünner Draht, der zwischen Sattel und Brücke einer Gitarre gespannt ist und beim Anschlagen schwingt, um Töne zu erzeugen.

Tremolo Eine spezielle Art von Gitarrenbrücke, die es ermöglicht, die Tonhöhe der Saiten durch Bewegung eines Hebels zu verändern.

Floyd-Rose Erfinder des gleichnamigen Tremolosystems, das in vielen E-Gitarren verwendet wird.

Stimmen einer Gitarre Der Prozess, bei dem die Spannung der Saiten angepasst wird, um die gewünschten Tonhöhen zu erreichen.

Stimmwirbel ein drehbarer Stift aus Metall oder Holz an Saiteninstrumenten, um den das Ende einer Saite gewickelt wird.

E | A | D | G | B | hohe E - Saite Die Namen der sechs Saiten einer Gitarre, von der tiefsten (E) bis zur höchsten (hohe E).

Bund Ein Metallstift, der quer über den Gitarrenhals verläuft und die Saiten in Abschnitte unterteilt, um verschiedene Töne zu erzeugen, wenn die Saite auf den Bund gedrückt wird.

MAUI Multi-platform App UI

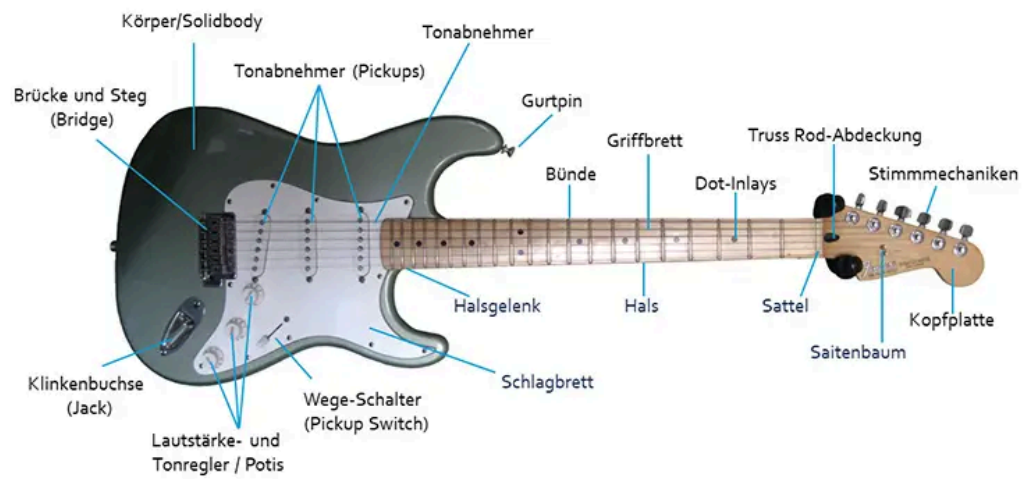


Abbildung 31: Begriffe einer Gitarre

Tabellenverzeichnis

Bibliografie

- [1] YouTube, „Quickly Tune A Floyd Rose Trem“. [Online]. Verfügbar unter: <https://youtu.be/z8ftAOFr4sg?t=239>
- [2] U. G. Forum, „How long does it take you to tune your Floyd Rose guitar?“. [Online]. Verfügbar unter: <https://www.ultimate-guitar.com/forum/showthread.php?t=1125303>
- [3] R. Schütz, „Anhänge zur Bachelorarbeit: Floyd-Rose-Tuner App“. Zugegriffen: 18. April 2026. [Online]. Verfügbar unter: https://github.com/Raphael2b3/bachelorthesis_floyd_rose_tuner
- [4] Wikipedia contributors, „Mersenne's laws“. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Mersenne%27s_laws
- [5] Wikipedia contributors, „Hookesches Gesetz“. [Online]. Verfügbar unter: https://de.wikipedia.org/wiki/Hookesches_Gesetz
- [6] Wikipedia-Autoren, „Hebel (Physik)“. [Online]. Verfügbar unter: [https://de.wikipedia.org/wiki/Hebel_\(Physik\)](https://de.wikipedia.org/wiki/Hebel_(Physik))
- [7] LEIFphysik, „Kombination von Federn oder Gummis“. [Online]. Verfügbar unter: <https://www.leifphysik.de/mechanik/kraft-und-das-gesetz-von-hooke/grundwissen/kombination-von-federn-oder-gummis>
- [8] Technikermathe.de, „Orthogonale Zerlegung von Vektoren“. [Online]. Verfügbar unter: <https://technikermathe.de/ma4-orthogonale-zerlegung-von-vektoren>
- [9] Wikipedia-Autoren, „Korrelationskoeffizient nach Bravais-Pearson“. Zugegriffen: 4. Mai 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/wiki/Korrelationskoeffizient_nach_Bravais-Pearson
- [10] Wikipedia-Autoren, „Orthogonale Regression“. [Online]. Verfügbar unter: https://de.wikipedia.org/wiki/Orthogonale_Regression
- [11] A. Cheveigné und H. Kawahara, „YIN, A fundamental frequency estimator for speech and music“, *The Journal of the Acoustical Society of America*, Bd. 111, S. 1917–1930, 2002, doi: 10.1121/1.1458024.
- [12] M. Stanek und T. Smatana, „Comparison of fundamental frequency detection methods and introducing simple self-repairing algorithm for musical applications“, in *2015 25th International Conference Radioelektronika (RADIOELEKTRONIKA)*, 2015, S. 217–221. doi: 10.1109/RADIO-ELEK.2015.7129013.
- [13] A. M. Noll, „Cepstrum Pitch Determination“, *The Journal of the Acoustical Society of America*, Bd. 41, Nr. 2, S. 293–309, 1967, doi: 10.1121/1.1910400.
- [14] J. W. Kim und J. P. Bello, „CREPE: A Convolutional Representation for Pitch Estimation“, in *Proceedings of the International Society for Music Information Retrieval Conference*, 2019. [Online]. Verfügbar unter: <https://arxiv.org/abs/1802.06291>
- [15] Wikipedia contributors, „Nyquist–Shannon sampling theorem“. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem
- [16] G. Vollmer, *Mobile App Engineering : eine systematische Einführung - von den Requirements zum Go Live /*, 1. Auflage. Heidelberg: : dpunkt.verlag, 2017. [Online]. Verfügbar unter: <http://vub.de/cover/data/isbn:9783864904219/medium/true/de/vub/cover.jpg>

- [17] Meta Platforms, Inc., „React Native: Learn once, write anywhere“. [Online]. Verfügbar unter: <https://reactnative.dev/>
- [18] Microsoft, „.NET Multi-platform App UI (.NET MAUI)“. [Online]. Verfügbar unter: <https://dotnet.microsoft.com/en-us/apps/maui>
- [19] Google, „Flutter: Build apps for any screen“. [Online]. Verfügbar unter: <https://flutter.dev/>